Exp no 3: Adversarial Search using Alpha-Beta Pruning

Experiment No.: 03

Experiment Title

Implementation of Adversarial Search using Alpha-Beta Pruning in Python.

Aim

To implement Adversarial Search using the Alpha-Beta Pruning

Objective

- To understand the concept of adversarial search.
- To study the Minimax algorithm and its limitations.
- To learn Alpha-Beta Pruning as an optimization technique.
- To implement Alpha-Beta Pruning using Python.
- To reduce the number of nodes evaluated in a game tree.

Theory

Adversarial Search is used in Artificial Intelligence for decision-making in competitive environments where two or more players have conflicting goals. Examples include chess, tic-tac-toe, checkers, and other strategic games.

The **Minimax Algorithm** is a popular adversarial search technique that assumes both players play optimally. One player, called **MAX**, attempts to maximize the utility value, while the other player, called **MIN**, attempts to minimize it.

Although Minimax guarantees the optimal move, it examines every node in the game tree, which can be computationally expensive for large trees.

Alpha-Beta Pruning improves the efficiency of the Minimax algorithm by eliminating branches that cannot affect the final decision.

Key Terms

- **Alpha (α):** Best value that MAX can guarantee.
- **Beta (β):** Best value that MIN can guarantee.
- **Pruning:** Ignoring branches that cannot influence the final result.

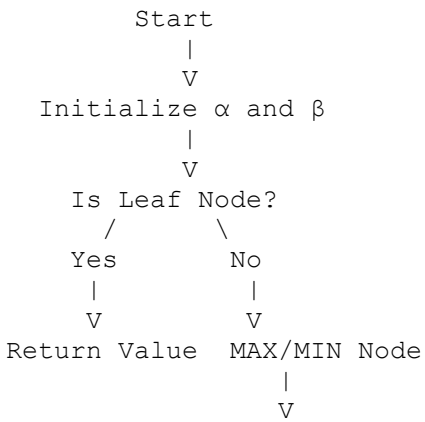
Advantages of Alpha-Beta Pruning

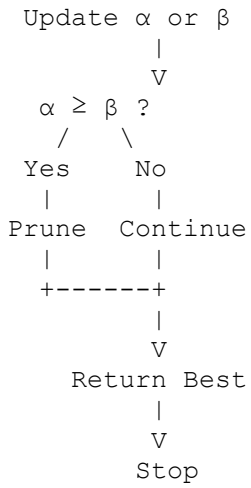
- Reduces the number of nodes explored.
- Improves search efficiency.
- Produces the same result as Minimax.
- Allows deeper game-tree exploration.

Algorithm

1. Start.
2. Initialize Alpha = −∞ and Beta = +∞.
3. If the current node is a leaf node, return its value.
4. For a MAX node:
    - Evaluate child nodes.
    - Update Alpha.
    - Select the maximum value.
5. For a MIN node:
    - Evaluate child nodes.
    - Update Beta.
    - Select the minimum value.
6. If Alpha ≥ Beta, prune the remaining branches.
7. Continue until the root node is evaluated.
8. Display the optimal value.
9. Stop.

Flowchart





Program

```
# Alpha-Beta Pruning

def alphabeta(depth, node, maxPlayer, values, alpha, beta):

    if depth == 2:
        return values[node]

    if maxPlayer:
        best = -1000

        for i in range(2):
            val = alphabeta(depth+1, node*2+i, False,
                            values, alpha, beta)

            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = 1000

        for i in range(2):
            val = alphabeta(depth+1, node*2+i, True,
                            values, alpha, beta)

            best = min(best, val)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best
```

```
values = [3, 5, 6, 9]

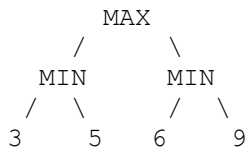
result = alphabeta(0, 0, True, values, -1000, 1000)

print("Optimal Value =", result)
```

Sample Output

```
Optimal Value = 6
```

Game Tree



Observation Table

Node Type	Values Examined	Selected Value
MIN (Left)	3, 5	3
MIN (Right)	6, 9	6
MAX (Root)	3, 6	6

Result

The Alpha-Beta Pruning algorithm was successfully implemented using Python. The algorithm efficiently reduced the number of nodes evaluated while producing the same optimal result as the Minimax algorithm. The optimal value obtained for the given game tree was **6**.

Applications

1. Chess programs.
2. Tic-Tac-Toe games.
3. Checkers and board games.
4. Strategic decision-making systems.
5. Automated game-playing agents.

Advantages

- Faster than Minimax.
- Reduces search space.
- Produces optimal results.
- Efficient for large game trees.

Limitations

- Performance depends on node ordering.
- Still expensive for extremely large trees.
- Requires additional memory for recursion.

Viva Questions

1. What is Adversarial Search?
2. What is the Minimax Algorithm?
3. What is Alpha-Beta Pruning?
4. What is the role of Alpha?
5. What is the role of Beta?
6. What is pruning?
7. How does Alpha-Beta Pruning improve Minimax?
8. What is a game tree?
9. What is a MAX node?
10. What is a MIN node?
11. What is the time complexity of Minimax?
12. What is the advantage of Alpha-Beta Pruning?
13. Can Alpha-Beta Pruning change the final result?
14. Name some applications of Alpha-Beta Pruning.
15. What is the optimal value obtained in this experiment?

Conclusion

The experiment successfully demonstrated Adversarial Search using Alpha-Beta Pruning. The algorithm efficiently searched the game tree, pruned unnecessary branches, and identified the optimal value while reducing computational effort.